ASTRA

Marco 5

Exclusão, erros e estados da tela

Gabriel Crispim · Eurípedes Brenno · Felipe

ASTRA / MARCO 5 / #31

Como implementamos a exclusão

ASTRA

Dividimos o fluxo entre o cartão, o módulo de API e a página que guarda as tarefas.

01 / NAVEGADOR
Cartão pede confirmação

02 / API
api.ts envia DELETE

03 / SVELTE
Página altera o $state

Enviar a exclusão para a API

src/lib/api.ts

```
// url: /tarefas/ID
const resposta = await fetch(url, {
  method: 'DELETE'
});
if (!resposta.ok) {
  throw new Error(...);
}
```

RESPOSTA HTTP

204

No Content: sucesso,
sem corpo de resposta.

resposta.ok verifica o sucesso. Em 204 não há corpo para converter com resposta.json().

O fetch envia o pedido; o json-server remove o registro do db.json. Nenhuma biblioteca nova foi instalada.

#31 / COMPONENTES E REATIVIDADE

Conectar o cartão à lista no Svelte

01 / CARTÃO	02 / API.TS	03 / CALLBACK	04 / SVELTE
Confirmar	Excluir	Avisar a página	Atualizar a lista

No TarefaCard, window.confirm abre a confirmação nativa. Se a pessoa cancelar, return encerra a função antes da requisição.

TarefaCard.svelte

```
if (!window.confirm(...)) return;
```

Callback é uma função passada por prop: a página entrega onexcluir ao cartão para receber o ID excluído.

#31 / DEMONSTRAÇÃO LOCAL

Testar a confirmação e a exclusão

Preparar a apresentação

A fazer · prioridade média

Excluir ×

Esta demonstração não acessa a API real.

window.confirm é a confirmação nativa do navegador. Cancelar impede o DELETE.

excluindo = true desabilita os controles durante o pedido, evitando envios repetidos.

Sucesso chama onexcluir(id). A página usa filter para produzir uma coleção sem aquele ID.

A coleção está em $state; $derived recalcula os filtros. O Svelte atualiza os cartões sem F5.

#32 / CARREGAMENTO

Iniciar a busca e mostrar a espera

src/routes/+page.svelte

```
let carregando = $state(true);

onMount(async () => {
  tarefas = await listarTarefas();
  // tratamento no slide seguinte
});
```

PRÉVIA DA LISTA / SIMULAÇÃO LOCAL

- Tarefas recebidas

`onMount` é a função do Svelte que roda após montar o componente no navegador. `$state(true)` começa indicando que a busca está em andamento.

A mensagem de espera evita confundir uma requisição em andamento com uma lista vazia.

Encerrar o carregamento no Svelte

`await` espera a resposta de listarTarefas. O resultado é guardado em tarefas.

`finally` roda no sucesso e na falha: carregando volta para false e encerra o aviso de espera.

`$state` torna o valor reativo: ao mudar, o Svelte atualiza o trecho da tela que lê esse valor.

Trecho do carregamento

```
try {
  tarefas = await listarTarefas();
} /* catch trata a falha */
finally {
  carregando = false;
}
```

No template, {#if carregando} mostra a espera enquanto o valor é true. Quando vira false, o Svelte avalia o próximo bloco.

#33 / TRATAMENTO DE ERRO NA LISTA

Capturar e exibir o erro da busca

api.ts / página

```
// api.ts detecta a falha:
if (!resposta.ok) throw ...;

// +page.svelte mostra a mensagem:
catch (e) {
  erro = 'Não foi possível...';
}
```

- Não foi possível carregar as tarefas.

Em api.ts, resposta.ok identifica HTTP 500. O throw envia a falha ao catch da página, que guarda a mensagem em erro, um $state.

O template exibe erro com role="alert". A mensagem está implementada; o botão “Tentar novamente” ainda não existe.

#32 + #33 / RENDERIZAÇÃO CONDICIONAL

Escolher o conteúdo com {#if}

```
{#if carregando}
  Aguardando a resposta
{:else if erro}
  A busca falhou
{:else if ...length === 0}
  Nenhum resultado
{:else}
  Exibir os cartões
```

SAÍDA DO TEMPLATE / SIMULAÇÃO LOCAL

- Carregando tarefas...

A ordem é: espera, erro, lista vazia e cartões. Só o primeiro bloco verdadeiro é exibido. O Svelte acompanha os valores em $state e troca o conteúdo dessa região.

A #34, apresentada a seguir, diferencia ausência de tarefas de ausência de resultados após os filtros.

#34 / ESTADO VAZIO · FELIPE

Distinguir lista vazia de busca sem resultado

Lista e quadro · após conferir carregamento e erro

```
{#if tarefas.length === 0}
  Nenhuma tarefa cadastrada ainda.
{:else if tarefasFiltradas.length === 0}
  Nenhuma tarefa encontrada com
  a busca e os filtros atuais.
{:else}
  <!-- renderizar cartões -->
{/if}
```

> Nenhuma tarefa cadastrada ainda.

A API respondeu sem itens: tarefas.length é zero. Essa condição vem antes da verificação dos filtros.

Os blocos condicionais do Svelte usam os dados existentes. No quadro, a coleção original vem de data.tarefas; o projeto mantém seu aviso de ausência de tarefas.

#35 / REGISTRO INEXISTENTE · FELIPE

Separar “não encontrado” de falha da API

- api.ts guarda o status; +page.ts interpreta

```
// ErroApi estende Error
catch (erro) {
  return {
    tarefa: null,
    naoEncontrada:
      erro instanceof ErroApi &&
      erro.status === 404
  };
}
```

> Tarefa não encontrada.

O load retorna naoEncontrada: true. O template escolhe a mensagem de registro inexistente.

---

load é o carregamento da rota no SvelteKit. A página recebe o resultado em data e usa {#if} para mostrar os dados ou a mensagem correta.

#36 / FALHA AO SALVAR

Preservar os campos quando salvar falha

Título da tarefa

Preparar a apresentação

Simulação local: nenhum dado é enviado à API.

Os campos usam `bind:value`. O `catch` muda só `erroEnvio`. Como `campos` não é zerado, o texto permanece.

TarefaFormulario.svelte

```
try {
  await aoEnviar({ ...campos });
} catch {
  erroEnvio = 'Não foi possível...';
} finally {
  enviando = false;
}
```

aoEnviar é recebido por prop: no cadastro chama criarTarefa; na edição, atualizarTarefa. finally libera o botão para outra tentativa.

#36 / FALHA AO EXCLUIR

Tratar o erro sem remover a tarefa

Preparar a apresentação

A tarefa permanece nesta simulação.

Simulação local: a API real não é acessada.

Se DELETE falhar, o catch mostra o erro. O finally muda excluindo para false e libera o botão.

TarefaCard.svelte

```
try {
  await excluirTarefa(tarefa.id);
  onexcluir(tarefa.id);
} catch {
  erroExclusao = 'Não foi
possível...';
} finally {
  excluindo = false;
}
```

onexcluir só executa depois do await bem-sucedido. Em falha, a página não recebe ordem para remover o cartão.

#37 / ACESSIBILIDADE

Conferir acessibilidade com teclado

Usamos Lighthouse no DevTools para apontar problemas e conferimos os controles pelo teclado, observando o foco.

Título da tarefa

Preparar a apresentação

Conferir tarefa

Demonstração local de campo e botão.

HTML no componente Svelte

```
<label for="titulo">Título</label>
<input id="titulo" />

/* CSS compartilhado */
:focus-visible {
  outline: 2px solid ...;
  outline-offset: 4px;
}
```

O label identifica o campo. :focus-visible mostra qual controle receberá a próxima ação do teclado.

Tab percorre os controles. Shift + Tab volta. Enter aciona o botão focado.

#37 / RESPONSIVIDADE E AUDITORIA

Conferir o layout em telas pequenas



As media queries do CSS reorganizam os mesmos cartões. Não criamos outra página para cada tamanho.

`1280 px · quatro colunas`

quadro/+page.svelte · CSS

```
.quadro-container { /* 4 colunas */
}

@media (max-width: 1100px) {
  /* 2 colunas */
}
@media (max-width: 640px) {
  /* 1 coluna */
}
```

CSS dentro do componente Svelte: o navegador aplica a regra conforme a largura da tela.

No PR #71, Brenno registrou o uso do Lighthouse e adicionou meta description em src/app.html, documento base do SvelteKit. Nenhum pacote novo.

INFRAESTRUTURA LOCAL / ARQUITETURA ATUAL

Como as partes do ASTRA se conectam

NAVEGADOR + SVELTEKIT · localhost:5173

Rotas e carregamento
`+page.svelte · +page.ts`

Componentes
`Cartão · formulário · filtros`

Estado reativo
`$state · $derived · stores`

Requisições HTTP
`api.ts / fetch`

API LOCAL · localhost:3000

json-server
`GET · POST · PATCH · DELETE`

db.json e tipos
`Dados · contrato TypeScript`

Rotas e carregamento

O SvelteKit cria as rotas pelas pastas de src/routes. +page.svelte apresenta a tela; +page.ts pode buscar os dados antes dela aparecer.

```
src/routes/
  tarefas/[id]/+page.ts
  tarefas/[id]/+page.svelte
```

Na lista geral, a busca começa em onMount no navegador. Em outras telas, usamos load, que também pode executar no servidor na primeira abertura.

Dois processos locais: npm run dev e npx json-server db.json. Hospedagem e configuração de produção ficam no Marco 6.